Scenario ID and Given Prompt: Scenario #1 – Search Functionality - Test the search feature

What is missing in the given prompt:

1. The given prompt is generic and contains only partial instruction.
2. It does not provide enough context of what kind of search functionality has to be tested.
3. The prompt is not clear on the input allowed, rule or constraints present as per the requirements, what is expected as results. For example, what will be the results when an edge case or a negative input is given
4. There is no example provided for the model to provide desired response or no persona assigned saying how the model should behave for the input given.

Weak Prompt Output from Chatgpt: Sure! What would you like to search for? I can test the search feature with a simple keyword, a specific topic, or a real-world search query.

Rewritten ICEPOT prompt output from Chatgpt:

Test Scenarios — Student Search Functionality

Application: School Teacher Portal Role: Senior QA Test Lead Total Scenarios: 10 Coverage: Positive, Negative, and Edge Cases

Test Scenario ID	High level classification	Prerequisite (if any)	Scenario Description	Test Steps	Expected Result	Test Owner
TS_SRCH_001	Positive	A student record exists with a known full name.	Verify search using the full name of an existing student.	1. Log in to the teacher portal. 2. Navigate to the student search page. 3. Enter the full name of an existing student. 4. Press Enter on	The matching student record is displayed. The record contains student name, parents' name, class, section, registration number, contact number, address,	<provide tester name>

				the keyboard.	attendance report, and current academic year's marks.	
TS_SRCH_002	Positive	Multiple student records exist with names containing the specified partial name.	Verify search using a partial student name.	1. Navigate to the student search page. 2. Enter a partial name that matches multiple students. 3. Click the search icon.	All matching student records containing the entered partial name are displayed. Results are sorted in ascending order of class.	<provide tester name>
TS_SRCH_003	Positive	A student record exists with a known registration number.	Verify search using an exact registration number.	1. Navigate to the student search page. 2. Enter a valid registration number. 3. Press Enter on the keyboard.	The correct student record associated with the registration number is displayed with all required student details. No unrelated student records are displayed.	<provide tester name>

TS_SRCH_004	Positive	A student record exists whose name contains a dot (.).	Verify search using the permitted dot special character.	1. Navigate to the student search page. 2. Enter a valid search term containing a dot, such as a student name with an initial followed by a dot, if supported by the test data. 3. Click the search icon.	The dot is accepted without a special-character validation error. Matching records are displayed according to the application's search-matching rules.	<provide tester name>
TS_SRCH_005	Positive	Multiple matching student records exist across different classes.	Verify that name-search results are sorted in ascending order of class.	1. Navigate to the student search page. 2. Enter a partial name that matches students in different classes. 3. Press Enter on	All matching student records are displayed in ascending order of class. Records belonging to lower classes appear before records belonging to higher classes.	<provide tester name>

				the keyboard. 4. Verify the class values in the returned list.		
TS_SRCH_006	Negative	No student record exists for the specified search term.	Verify search behavior when no matching student record exists.	1. Navigate to the student search page. 2. Enter a valid-format name or registration number that does not exist. 3. Click the search icon.	No student records are displayed. An appropriate “No records found” message is displayed, and the application remains stable.	<provide tester name>
TS_SRCH_007	Negative	The search field is available.	Verify validation when the search input contains unsupported special symbols other than a dot (.).	1. Navigate to the student search page. 2. Enter unsupported special symbols, such as @, #, or $.	An appropriate validation error prompts the teacher not to use unsupported special symbols. The search is not executed with invalid input.	<provide tester name>

				3. Click the search icon.		
TS_SRCH_008	Negative	The search field is available.	Verify search behavior when the input field is left empty.	1. Navigate to the student search page. 2. Leave the search field empty. 3. Press Enter on the keyboard. 4. Repeat by clicking the search icon with the field empty.	The application handles both actions consistently. An appropriate validation message is displayed, or the search is not triggered. All student records must not be exposed unintentionally.	<provide tester name>
TS_SRCH_009	Edge case	A student record exists with a name or registration number that includes leading or trailing spaces in the search input.	Verify search behavior when the search term contains leading or trailing whitespace.	1. Navigate to the student search page. 2. Enter a valid student name or registration number with leading and/or trailing spaces.	The application handles surrounding whitespace consistently, preferably by trimming it before searching. The correct matching record is displayed if whitespace normalization is part of the	<provide tester name>

				3. Click the search icon.	specified behavior.	
TS_SRCH_010	Edge case	Test data includes a student whose name has multiple words and another student with a similar or overlapping name.	Verify search behavior for case variations and overlapping partial names.	1. Navigate to the student search page. 2. Search for an existing student's name using different letter casing. 3. Repeat using a partial name shared by multiple students. 4. Verify the returned records and their ordering.	Search results follow the application's defined case-sensitivity rules. For partial-name searches, all relevant matching records are returned without unintended omissions or duplicate entries, and results are sorted in ascending order of class.	<provide tester name>

QA Lead Notes

- Search triggers: Validate both keyboard Enter and search-icon click behavior.
- Input validation: The dot (.) is permitted; other special symbols should trigger the defined validation message.
- Search results: Full-name and registration-number searches should identify the correct records; partial-name searches should return all relevant matches.
- Data integrity: Verify all eight categories of student information specified in the requirements are displayed correctly.

- Sorting: Confirm ascending class order for name and partial-name search results.
- Clarifications recommended before execution:
    1. Define whether searches are case-sensitive.
    2. Confirm whether leading and trailing spaces should be trimmed.
    3. Define expected behavior for blank input and inputs containing only a dot.
    4. Define how records within the same class should be ordered, if a secondary sort is required.

Observations on Improvements seen:

The weak prompt fetched only a generic response where the model asked further questions. This will result in back and forth conversation between model and the user resulting in increased token usage that impacts cost and budget. Also this might result in incorrect results or deviated response over time.

But, when ICEPOT prompt structure is provided with clear set of details, the model generated a well structured and intended valid response for use. In addition it also generated a Notes with a list of points that the tester needs to clarify or make aware of to ensure quality product.